

ZTracker — Quick User Guide

Fiji / ImageJ plugin · two tools: 3D Z-Coordinate Extractor + Z-Projection maker

What this plugin does

ZTracker takes your **2D cell tracks** (from TrackMate or another tracker) and adds the **Z (depth) coordinate** to every detection, turning 2D tracks into 3D tracks.

It reads depth from special **Z-origin projection TIFF images**, where each pixel's value is a code that looks up a real depth in micrometers from a `.json` file. For each tracked point, the plugin looks at the pixel(s) under it, converts them to depth, and saves the result.

What you need before you start

File	What it is
Z-mapping JSON	A <code>.json</code> file that maps pixel codes to depths in μm , e.g. <code>{"0": -600.0, "1": -599.0, ...}</code> .
Z-origin TIFF folder	A folder of TIFF images (one per time frame), 16-bit or 32-bit. These are the depth-coded projection images.
Tracking CSV	Your tracks exported from TrackMate (or another tracker). Must contain X, Y, Frame, and Track ID columns.

Don't have the **Z-mapping JSON** and **Z-origin TIFF folder** yet? The companion tool **Z-Projection + Origin Map** makes both from a raw Z-stack — see the last section.

Opening the plugin

In Fiji, go to **Plugins > ZTracker > 3D Z-Coordinate Extractor**. The plugin runs as a simple 6-step wizard. You can move, resize, and read the Fiji **Log** window at any time while a step is open — it stays usable.

The 6 steps

1 Pick your 3 input files

Use the `...` browse buttons to select the **Z-mapping JSON**, the **Z-origin TIFF folder**, and the **tracking CSV**. Click OK.

2 Confirm the CSV format

The plugin asks how to read your CSV (header row, rows to skip, default cell radius). TrackMate exports have 3 extra info rows under the header — the defaults usually handle this. If you're unsure, leave the defaults and continue.

3 Confirm the columns

The plugin auto-detects which columns are X, Y, Frame, and Track ID and shows its guess. Check they look right and correct any that are wrong, then continue.

4 Line up the frames (offset)

Trackers often number frames starting at 0, while TIFF files often start at 1. This step lets you shift the CSV frames so they match the TIFF files. A suggested offset is already filled in (**+1 is the most common correct value**).

As you change the number, the box shows a live **per-track verdict**, and a full table is printed to the Fiji **Log** so you can check that every track lands on a real TIFF frame. When the box says all tracks map correctly, confirm.

Why it matters: a wrong offset silently pairs each track point with the wrong image, giving wrong depths. Take a moment to confirm the verdict looks good.

5 Choose how to sample depth

Pick how the plugin reads pixels under each detection:

- **Sampling method:** *Radius* (average a small disk — good default), *4-Neighbor*, or *Single Pixel*. You can also choose **All** to run every method.
- **Aggregation:** *Median* (robust to odd pixels — recommended) or *Mean*. (Disabled automatically for Single Pixel, since there's only one value.)
- **Pixel convention:** leave on *Corner* (the default) unless you have a specific reason to use *Center*.

Not sure? Radius + Median + Corner is a solid default for most users.

6 Choose output folder & formats

Pick where to save results and tick the formats you want (see the next section). Click OK to run.

What you get out

Format	What it's for
.npy files (<code>tracks_2D/</code> , <code>tracks_3D/</code>)	For Python analysis/plotting. 3D files hold [X, Y, Z, T].
Results Table CSV	Open in Fiji via <code>Analyze > Import > Results...</code> , or in Excel.
ROI set (.zip)	Open in Fiji's ROI Manager (<code>More >> Open...</code>) to overlay points on your image. Optional XZ/YZ side-view sets are also available.
export_report.txt	A plain-text summary of every track — how many points were kept or dropped, and why. Check this if numbers look off.

X and Y are in **pixels**, Z is in **micrometers**, and T is the frame number.

Good to know

- **No track is thrown away.** If one point is bad (e.g. its frame is missing or it falls outside the image), only that single point is dropped — the rest of the track is still exported.
- **Dropped points leave a gap** in the frame numbers rather than renumbering — this is intentional.
- **Check the Log window** after running. It reports how many points were extracted and how many were skipped, and why.
- If you picked **All** in Step 5, each method gets its own sub-folder inside your output folder.

Quick troubleshooting

Symptom	Likely fix
Lots of points show "missing frame"	Revisit Step 4 — the frame offset is probably wrong (try +1 or 0).
Lots of points show "out of bounds"	Check your CSV X/Y match the TIFF image size, and check the pixel convention in Step 5.
Columns detected wrong	Fix them manually in Step 3 .
Plugin not in the menu	Restart Fiji, or use <code>Help > Refresh Menus</code> .
Nothing seems to export	Make sure at least one format is ticked in Step 6 , and read <code>export_report.txt</code> .

Companion tool: Z-Projection + Origin Map

This second tool **makes** the two projection inputs the extractor needs, starting from a raw Z-stack. Open it via **Plugins > ZTracker > Z-Projection + Origin Map**.

What it needs: a **dataset folder** whose sub-folders are named by their depth in μm (e.g. `-300` , `-299` , ...), each holding one TIFF per time frame (same filename across depths):

```
dataset/ ├── -300/ frame_0001.tif, frame_0002.tif, ... ├── -299/ frame_0001.tif, ... └ ...
```

In the dialog you pick the input folder, the output folder, and:

- **Scope** — *Single dataset* (the folder above) or *Batch* (a folder holding many such datasets).
- **Projection** — *Max-Z* (keeps the **brightest** pixel at each position — the usual choice for fluorescence) or *Min-Z* (keeps the **darkest**).

What it makes (for each dataset, under a `max_z` or `min_z` folder):

Output	What it is
z_origin/ (16-bit) and z_origin_32bit/ (32-bit)	The depth-coded projection TIFFs — point the extractor's Z-origin TIFF folder here.
z_layer_mapping.json	The pixel-code → depth map — this is the extractor's Z-mapping JSON .
raw/	A plain 8-bit projection picture for looking at (the extractor ignores it).

Then just run the extractor (Tool 1) and point it at the `z_origin` folder and the `z_layer_mapping.json` this tool produced.

The 16-bit and 32-bit depth images hold the same information — use 16-bit unless you have more than ~65,000 Z-layers (essentially never), in which case use the 32-bit set.

For full technical detail, see the project README.